PROJETO TASKTRACKER: FASE 2 (DESAFIO ETAPA INTERMEDIÁRIA)

📋 1. VISÃO GERAL DO PROJETO

O Bootcamp II é uma jornada de aprendizagem prática e cumulativa focada em Engenharia de Software, Versionamento Profissional de Código e Desenvolvimento de Sistemas. A jornada totaliza 75 horas de carga horária e é dividida em três fases interligadas:

1. **Fase 1 — Entrega Inicial (10,0 pts):** Planejamento Lógico, Arquitetura de Dados, Decomposição do Problema, Algoritmo em Português e Mapeamento para Git/GitHub. *(Concluída)*
2. **Fase 2 — Entrega Intermediária (15,0 pts):** Estruturação do Repositório, Versionamento com Git/GitHub, Implementação da Lógica em Python e Validação das Regras de Negócio. *(Etapa Atual)*
3. **Fase 3 — Entrega Final (25,0 pts):** Empacotamento da Aplicação, Containerização com Docker, Testes e Deploy.

🎯 2. OBJETIVO DA FASE 2 (ETAPA INTERMEDIÁRIA)

O objetivo central desta etapa é migrar o planejamento teórico desenhado na Fase 1 para um ambiente de desenvolvimento real. O estudante assume o papel de Desenvolvedor(a) de Software e deve:

- Criar e versionar a estrutura do projeto utilizando comandos do **Git**.
- Publicar a aplicação no GitHub mantendo a rastreabilidade do histórico de desenvolvimento (commits).
- Construir a aplicação CLI (Interface de Linha de Comando) em Python 3, garantindo o tratamento de dados e as regras de negócio definidas no algoritmo.
- Apresentar a solução técnica em um vídeo demonstrativo de 5 minutos no YouTube.

📁 3. ARQUITETURA E ESTRUTURA DO REPOSITÓRIO

A estrutura de arquivos e diretórios do projeto no repositório local e remoto no GitHub deve seguir obrigatoriamente a árvore planejada na Fase 1:

Plaintext

```
tasktracker-projeto/
├── README.md            # Documentação principal da aplicação e instruções de uso
├── .gitignore           # Regras para ignorar arquivos temporários do Python e ambiente virtual
├── docs/
│   └── planejamento_logico.pdf # PDF com o planejamento lógico e arquitetural entregue na Fase 1
└── src/
    └── main.py          # Código-fonte principal da aplicação TaskTracker em Python
```

Detalhamento dos Arquivos:

- **README.md:** Arquivo em formatação Markdown contendo o título do projeto, descrição funcional, tecnologias utilizadas, guia de instalação e execução.
- **.gitignore:** Garante as boas práticas de software, ignorando pastas automáticas como __pycache__/, arquivos .pyc e diretórios .venv.
- **docs/planejamento_logico.pdf:** Documento original da Fase 1 mantido para rastreabilidade do projeto cumulativo.
- **src/main.py:** Contém o código-fonte em Python com o loop do menu, funções de cadastro, listagem e tratamento de entradas.

💻 4. REQUISITOS DE IMPLEMENTAÇÃO EM PYTHON (src/main.py)

A implementação deve traduzir a "receita lógica" da Fase 1 em código funcional e bem estruturado:

A) Interface e Menu de Navegação

A aplicação deve operar via linha de comando, oferecendo um menu contínuo com as opções:

1. **Cadastrar nova tarefa**
2. **Visualizar tarefas cadastradas**
3. **Sair da aplicação**

B) Estrutura de Dados e Regras de Negócio

Cada tarefa deve ser representada como um objeto/dicionário contendo os seguintes campos e regras de validação:

- **Título (String):** Obrigatório. O sistema deve rejeitar entradas vazias ou compostas apenas por espaços em branco, solicitando nova digitação até que um título válido seja informado.
- **Descrição (String):** Detalhamento livre da tarefa.
- **Prioridade (String):** Validação estrita. Aceita exclusivamente as opções "Alta", "Média" (ou "Media") e "Baixa". Entradas fora desse padrão devem gerar mensagens de erro e repetição do prompt.
- **Data Limite (String):** Registro da data planejada para conclusão (formato livre ou DD/MM/AAAA).
- **Status (String):** Definido automaticamente como "Pendente" no momento do cadastro.

C) Listagem e Exibição

- **Lista Vazia:** Caso nenhuma tarefa tenha sido cadastrada, o programa deve exibir uma mensagem clara informando que não existem registros no momento.
- **Lista Preenchida:** Deve iterar sobre a lista de tarefas e exibir todas as propriedades organizadas e legíveis no terminal.

🔀 5. DIRETRIZES DE VERSIONAMENTO COM GIT E GITHUB

O histórico deve demonstrar a evolução do trabalho:

1. **Inicialização do Projeto (git init):** Criação do repositório e adição da estrutura básica de pastas.
2. **Commit Inicial (git commit):** Registro do README.md, .gitignore e documento da Fase 1 (docs/planejamento_logico.pdf).
3. **Commit de Desenvolvimento:** Implementação da estrutura principal em src/main.py (menu e loop principal).
4. **Commit de Regras e Validações:** Adição das funções de validação de dados (título não vazio e prioridades).
5. **Publicação Remota (git push):** Vinculação do repositório remoto no GitHub (origin main).

📹 6. REQUISITOS TÉCNICOS E ROTEIRO DO VÍDEO (YOUTUBE)

A entrega inclui obrigatoriamente a apresentação técnica gravada:

Regras Técnicas Inegociáveis:

- **Formato:** Proporção 16:9 (Horizontal). Proibido YouTube Shorts (Vertical).
- **Presença:** Rosto do estudante visível obrigatoriamente durante a explicação (webcam ligada) e voz própria sem uso de inteligência artificial generativa.
- **Duração:** 5 minutos.
- **Visibilidade:** Publicado no YouTube no modo **Público** ou **Não Listado**.

Roteiro Sugerido para Apresentação:

1. **Introdução (30s):** Nome completo, curso, polo (Asa Norte ou Taguatinga) e contexto da Fase 2 do Bootcamp II.
2. **Apresentação do GitHub (1m30s):** Mostrar o repositório publicado, a árvore de pastas, o arquivo README.md e o histórico de commits.
3. **Demonstração Prática do Código (2m):** Executar o arquivo src/main.py no terminal:
    - Forçar um erro de validação (ex: cadastrar título vazio ou prioridade inválida) para mostrar a regra funcionando.
    - Cadastrar uma tarefa com sucesso.
    - Exibir a lista de tarefas atualizada.
4. **Encerramento (30s):** Considerações finais e próximos passos para a Fase 3 (Docker).

📑 7. CRITÉRIOS DE AVALIAÇÃO (BAREMA DA FASE 2)

A pontuação total de **15,0 pontos** da Entrega Intermediária está distribuída conforme abaixo:

Dimensão Avaliada	Critérios Detalhados	Pontuação
Estrutura Git & GitHub	Criação do repositório no GitHub, presença da árvore completa de pastas/arquivos alinhada à Fase 1, e histórico com múltiplos commits coerentes.	**4,0 pts**
Código Python e Regras	Funcionamento sem erros do menu, fidelidade ao algoritmo da Fase 1, aplicação estrita das validações (título e prioridade) e exibição das tarefas.	**6,0 pts**
Apresentação em Vídeo	Cumprimento das regras técnicas (rosto em câmera e voz, 16:9, link ativo) e clareza na explicação prática do GitHub e execução do terminal.	**5,0 pts**
TOTAL		**15,0 pts**